

Supply, Demand and Policy in New York’s Ridehail Market: A PySpark MapReduce Analysis of 462 Million Trips

MIT 805 Big Data — Part 2: MapReduce & Visualisation

Gishon K. Gwenzi (21731056)

Letsoba Savannah Mabe (20582995)

University of Pretoria | 8 October 2026

Repository: <https://github.com/caie007/savannahgishon>

1 Processing objective and analytical question

New York’s high-volume for-hire vehicle (HVFHV) market moved 461,855,376 trips across 2024 and 2025. Part 1 established, on 2025 data alone, that passenger wait peaks at 04:00 and troughs at 10:00 — the inverse of the demand profile — implying that service quality is governed by driver supply rather than road congestion. It also showed that the central business district (CBD) congestion fee introduced on 5 January 2025 is a flat \$1.50 charge, and therefore proportionally heavier on low-fare trips.

Part 2 tests both propositions at scale and joins them into one question:

Where, when and for whom does New York’s ridehail market fail to match supply to demand — and did congestion pricing change that?

Three sub-questions follow. **(i)** Did CBD-bound volume fall after the charge, relative to trips that never enter the zone, and beyond any pre-existing trend? **(ii)** Who bore the cost of a flat fee? **(iii)** Which zones and hours are persistently under-served, and does supply density explain it?

Answering these requires a full pass over 24 months of trip records — roughly 107 GB uncompressed — which is not tractable on a single-threaded framework. The analysis is therefore implemented as MapReduce-style distributed processing in PySpark.

2 Data and distributed architecture

The working set is 24 monthly Parquet files (2024-01 to 2025-12), 11.1 GB compressed. After the Part 1 cleaning rules — positive distance and duration, duration ≤ 6 h, non-negative fare, dropoff after pickup, and both endpoints among the 265 mapped taxi zones — 229,391,984 records remain for 2024 and 232,463,392 for 2025.

A single distributed pass reduces these 461,855,376 rows to an aggregate of 73,973,527 rows keyed by (PU zone, DO zone, year), storing *sums* rather than means so that any later regrouping re-aggregates correctly. Every subsequent result in this report is computed from that aggregate. This pre-aggregation pattern is the practical core of the work: it converts a problem that does not fit in memory into one that does, and it is the shared layer from which both authors’ threads are derived.

2.1 Deriving the charged zone set from the data

The charged area was not hard-coded. In 2025 the `cbd_congestion_fee` column records which trips were charged, so a map–shuffle–reduce over destination zone yields the empirical charge rate per zone. The distribution is strongly bimodal (Appendix Fig. A1), so a threshold of 0.80 is not delicate: it identifies **38 CBD zones**, which were then applied backwards to 2024 to label trips that *would have been* charged. A ring of 32 Manhattan zones immediately outside the charged area was labelled *boundary* to test for displacement.

3 MapReduce implementation

3.1 Mapping and transformation

Each record is projected to the columns the analysis needs and enriched with derived fields: `year`, `hour`, a boolean `cbd_trip` from the derived zone set, and `wait_s = pickup_datetime - request_datetime`, with a `wait_valid` guard excluding the 1.15 % of records whose pickup precedes the request (a telematics clock-drift artefact identified in Part 1). All of this is lazy: the physical plan in Appendix A shows Catalyst collapsing six chained projections into two whole-stage code-generated scans.

3.2 Shuffle and grouping

Aggregation by zone and hour requires rows sharing a key to meet on one executor, so Spark repartitions by hash of the grouping key. This is the `Exchange hashpartitioning` node in the physical plan and the only point at which data crosses the network. For the main aggregation the exchange moved 7.2 GiB — 6 % of the 120.5 GiB read from disk, the reduction coming from column pruning pushed into the Parquet reader and from partial aggregation applied before the shuffle.

3.3 Reduction and aggregation

We implemented the identical computation — trip count and mean wait by (PU zone, hour) — twice, to make the execution models comparable on like terms (Table 1; see also Fig. A5).

Both produced 6,143 identical keys, with zero difference in counts and mean waits agreeing to floating-point precision. The RDD path makes the three phases explicit: `map` emits $((\text{zone}, \text{hour}), (1, \text{wait}))$, `reduceByKey` combines

Table 1: Two implementations of one aggregation, 1,964,006 rows, 8 partitions.

Implementation	Runtime	Shuffle write	Keys produced
RDD <code>map</code> $\rightarrow$ <code>reduceByKey</code>	12.7 s	558.3 KiB	6,143
DataFrame <code>groupBy</code> $\rightarrow$ <code>agg</code>	1.9 s	1,061.4 KiB	6,143

locally on each partition before shuffling — the exact function of a Hadoop Combiner — and a final `map` converts partial sums to means. The stage metrics quantify that combining: the `reduceByKey` stage read 166.7 MiB and wrote only 558.3 KiB of shuffle, a **306-fold reduction** before any data crossed the network.

The decisive observation is that the RDD path shuffled *less* data than the DataFrame path yet ran 6.56 times slower. The gap is therefore not shuffle volume; it is the process boundary. Python RDD lambdas require every row to be serialised out of the JVM into a Python worker and back, whereas DataFrame operations are compiled to JVM bytecode by Tungsten and never leave it. This was not merely a performance difference: attempting the RDD benchmark over the full 2025 set caused Python worker handshake timeouts, which is the same cost appearing as a failure rather than a delay.

4 Distributed execution evidence

Table 2: Spark UI metrics for the full run (application MIT805-Part2-HVFBV, Spark 3.5.1, Java 17.0.20.1, local [8], 32 GB driver).

Metric	Value	Metric	Value
Tasks completed / failed	849 / 0	Total input read	120.5 GiB
Cores	8	Shuffle read / write	7.2 GiB / 7.2 GiB
Cumulative task time	44 min (GC 34s)	Disk used	28.2 GiB
Jobs / SQL queries	49 / 15	Shuffle partitions (AQE)	96
Stages completed / skipped	50 / 30	Wall clock	39 min

Three observations from Table 2 bear directly on the execution model. First, **30 of 80 stages were skipped** because Spark reused shuffle output already materialised by earlier jobs; a chained MapReduce pipeline has no equivalent and would recompute from input each time. Second, adaptive query execution coalesced the configured 256 shuffle partitions down to **96** at runtime based on observed partition sizes. Third, the cached intermediate frame reports 26.1 GiB on disk and 0 B in memory against 19 GiB of storage memory: it did not fit, and Spark degraded to disk rather than failing.

4.1 Spark’s DAG versus Hadoop’s Map $\rightarrow$ Shuffle $\rightarrow$ Reduce

Hadoop MapReduce executes one fixed Map $\rightarrow$ Shuffle $\rightarrow$ Reduce cycle per job and materialises output to HDFS between jobs, replicated three times by default. Spark instead compiles the whole computation into a DAG of stages whose boundaries fall only at shuffles, pipelining everything in between in memory. Four concrete contrasts from this run:

- **Intermediate I/O.** The 7.2 GiB that crossed our single shuffle would, under Hadoop, additionally be written to and re-read from HDFS between every stage — roughly 21.6 GiB of extra write traffic at default replication, none of which Spark incurs.
- **Automatic combining.** Catalyst placed a partial `HashAggregate` before the exchange and a final one after it. Hadoop requires a Combiner to be written by hand to achieve the same effect.
- **Join strategy.** The 265-row zone lookup was broadcast to every task, eliminating the shuffle of the large side entirely. Classical MapReduce has no broadcast join and must shuffle both inputs.
- **Lazy, replanned execution.** Nothing ran until an action, and AQE re-planned partitioning mid-flight. MapReduce is eager and fixed at submission.

5 Results

5.1 Thread A — Congestion pricing did less than the headline suggests

A naive year-on-year comparison looks dramatic: CBD-bound trips fell 4.39 % (79,897,901 to 76,390,002) while non-CBD trips rose 4.40 %, a difference of **−8.79 percentage points**. The CBD share of all trips fell from 34.83 % to 32.86 %.

That comparison is misleading, and Fig. 1(b) shows why. The CBD share was *already* falling through 2024 at -0.103 percentage points per month, before any charge existed. There is no discontinuity at the policy date: the share in January 2025 was 0.21 pp *above* December 2024, within the ordinary month-to-month variation of -0.199 pp observed across 2024. Extrapolating the 2024 trend predicts a 2025 mean of 33.60 % against an actual 32.88 % — a residual of only -0.72 pp, and the 2025 slope (-0.125 pp/month) is only marginally steeper than 2024’s.

Two further results corroborate a weak volume effect. Trips terminating in the boundary ring grew 0.79 % year-on-year against 5.26 % for outer zones (Fig. A2): had riders been avoiding the charge by stopping short of the boundary, the ring would have gained disproportionately, and it did not. Meanwhile CBD mean fare rose from

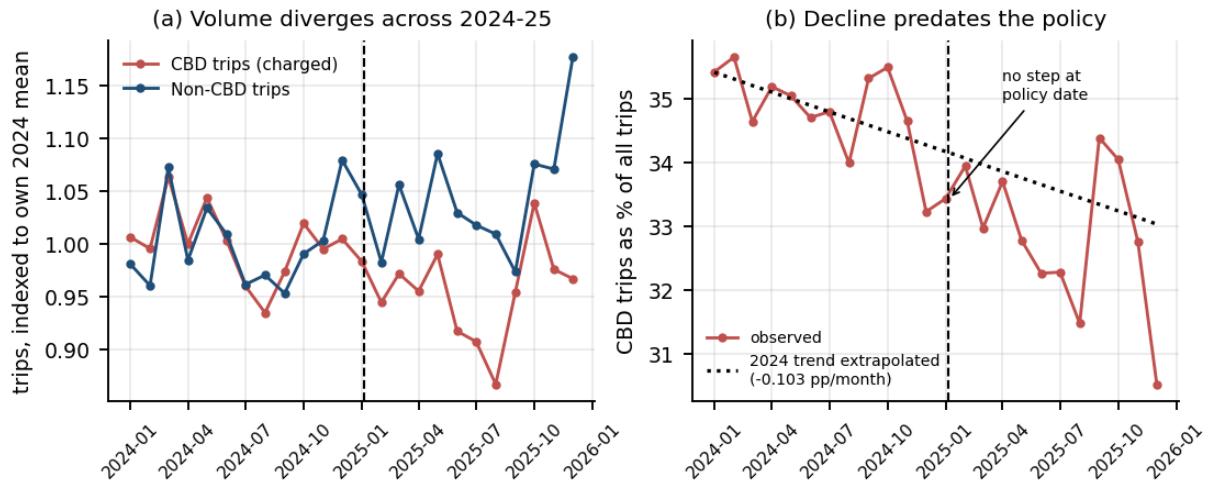


Figure 1: Trip volume by charge status. The divergence in (a) is real, but (b) shows most of it predates the 5 January 2025 charge.

\$31.24 to \$33.57 (+7.5 %) while non-CBD fare was flat (\$20.62 to \$20.65). The \$1.50 charge accounts for roughly 64 % of that \$2.33 increase, so the cost was passed to riders rather than suppressing the trip.

5.2 Who paid

Conditional on being charged, the flat fee is regressive (Fig. 2). Against mean borough fares it represents 7.98 % of a Bronx fare but only 5.17 % of a Manhattan one — a 1.54× spread, falling hardest on the lowest-fare riders. In aggregate the incidence is reversed: Manhattan pickups generated 83.1 % of the \$117,124,706 collected in 2025, because 76.3 % of Manhattan trips are charged against 3.7 % of Bronx trips. Both statements are true and they answer different questions: Manhattan funds the scheme, but an outer-borough rider who does cross the cordon pays half again as much of their fare for the privilege.

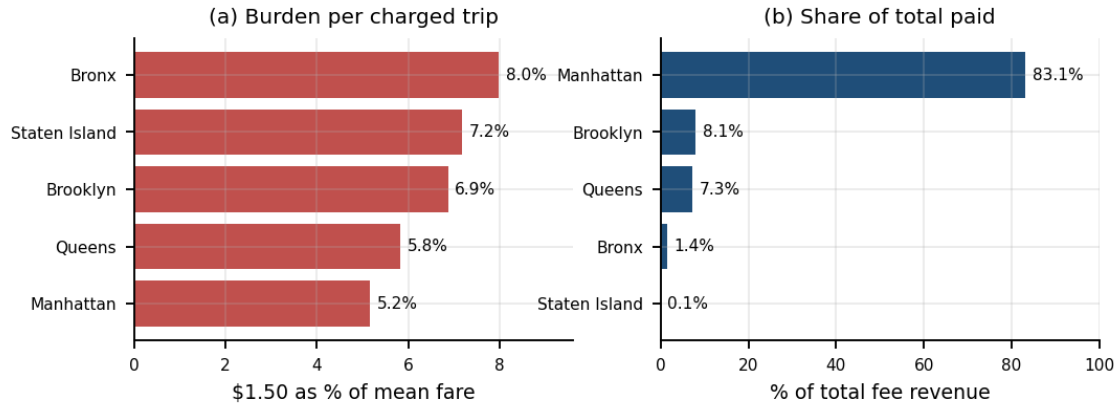


Figure 2: Flat-fee incidence. Per charged trip the burden is regressive (a); in aggregate Manhattan funds the scheme (b).

5.3 Thread B — Service quality is a function of supply density

Across 5,898 zone-hour cells we computed a *mismatch index*: mean wait in a zone-hour divided by the citywide mean wait for that same hour. Normalising by hour is essential — without it the analysis merely rediscovers that 04:00 is slow everywhere. The index reaches 2.35 at its worst, and its 90th percentile is 1.297.

The correlation between zone trip density and mean wait is **negative at every one of the 24 hours** (Fig. 3), from -0.24 at 08:00 to -0.617 at 20:00, with an overall $r = -0.472$. Busier zones are served *faster*, at all times of day. This confirms Part 1’s inference directly: the binding constraint is the density of available drivers, not road congestion. If congestion dominated, the sign would invert at peak hours, and it never does.

The same mechanism governs shared-ride matching. The match rate ranges from 70.9 % at 23:00 to 39.4 % at 11:00 and tracks request volume with $r = 0.59$ (Fig. A3): thin demand cannot be pooled, so passengers who accept a discounted shared fare at midday ride alone three times in five.

Persistence matters more than any single bad hour. Filtering to zone-hours above the 90th-percentile mismatch and counting distinct bad hours per zone yields **74 persistent service deserts**, four of which are above threshold in all 24 hours (Fig. 4, Fig. A4). They are overwhelmingly peripheral — 23 in Queens, 18 in Staten Island, 18 in Brooklyn — and concentrated in the Rockaway peninsula and southern Staten Island. Hammels/Arverne is worst at a mean index of 1.61, meaning its riders wait 61 % longer than the citywide norm for the same hour, every hour of the day.

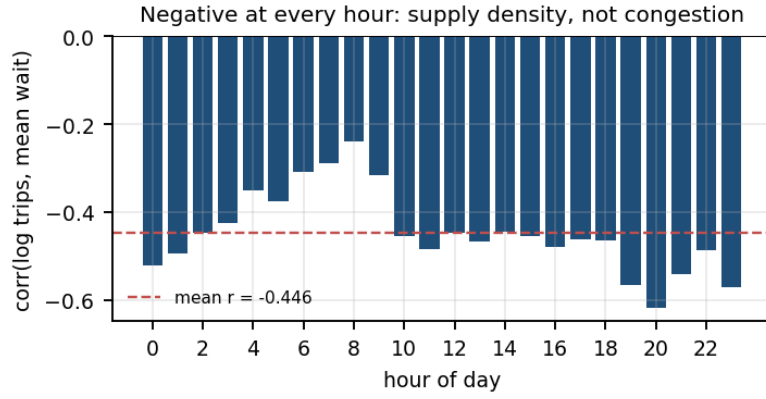


Figure 3: Correlation between zone trip density and mean wait, by hour.

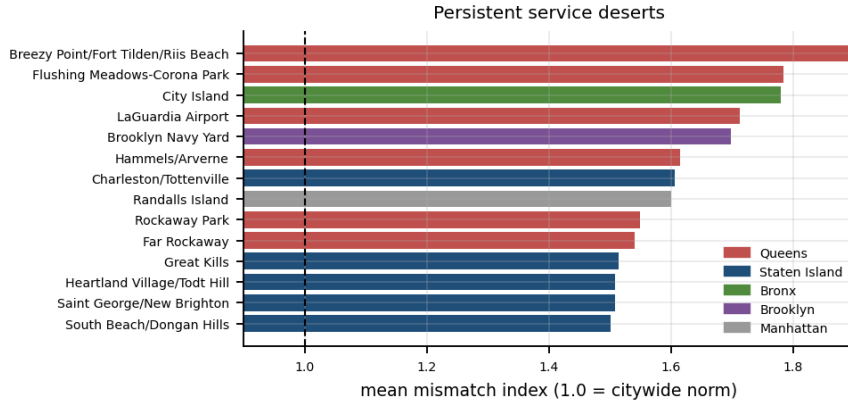


Figure 4: Zones above the 90th-percentile mismatch index for the greatest number of hours. Colour denotes borough.

5.4 Where the two threads meet

The threads converge on one number. Between 2024 and 2025 mean wait in the CBD *improved* from 272.0s to 269.2s (−1.0%), while mean wait everywhere else *deteriorated* from 281.0s to 294.1s (+4.7%). Congestion pricing did not materially remove trips from the cordon. What it appears to have done is redistribute driver supply — and given that service quality is determined by supply density at every hour of the day, a marginal gain inside the cordon was accompanied by a loss four times larger outside it, borne by the same peripheral zones already identified as service deserts.

6 Business and societal interpretation

For the TLC and the MTA, the headline finding is that the fee raised \$117,124,706 in its first year with no volume reduction distinguishable from the pre-existing trend. If the policy objective was revenue, it succeeded; if it was modal shift away from for-hire vehicles entering the CBD, the evidence here does not support it. The −0.72 pp residual against the 2024 trend is the honest upper bound on the volume effect, and even that is not separable from other 2025 changes.

For equity regulators, the flat structure produces a $1.54\times$ spread in proportional burden between the lowest- and highest-fare boroughs. A fee scaled to fare, or a rebate for trips originating in the zones identified in Fig. 4, would raise comparable revenue with materially less regressivity — and the dataset makes the trade-off directly computable.

For outer-borough riders, the service-desert result has a sharper implication than slow service. Part 1 showed these are the zones where ridehail behaves as transit substitution rather than discretionary travel. They are now shown to be both the worst-served at every hour and the places where waits deteriorated most during the policy year. A rider in Hammels/Arverne waits 61% longer than the city norm with weaker rail alternatives than a rider in Midtown.

For operators, the negative density–wait correlation at every hour is a positioning signal: the marginal value of an additional driver is highest exactly where current density is lowest. The 74 zones in Fig. A4 are a concrete target list, and the midday shared-match collapse to 39.4% identifies when pooling incentives fail.

Limitations. Three bound these conclusions. *First, identification* — 2024 and 2025 differ in many ways besides the fee, and we report association with an explicit pre-trend adjustment, not causation. *Second, survivorship* — the data records only matched, completed trips, so a service desert cannot be distinguished from suppressed demand; riders who stopped requesting are invisible, which means our mismatch index understates the problem. *Third, no entity identifiers* — without driver or vehicle keys, the supply redistribution we infer from wait times cannot be confirmed by observing drivers directly.

Appendix

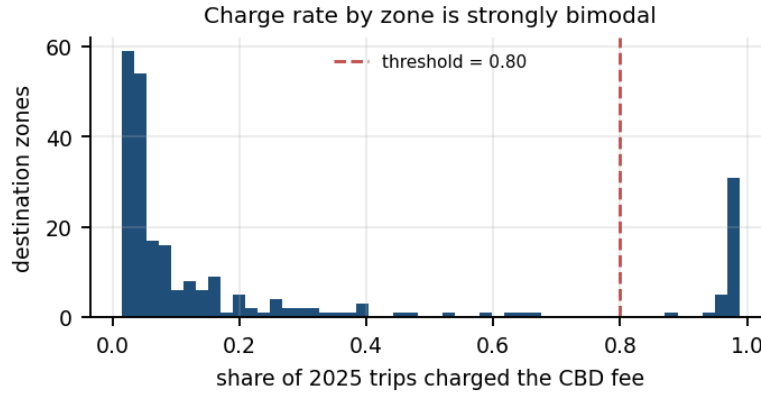


Figure A1. Empirical charge rate by destination zone, 2025. The distribution is strongly bimodal — 178 zones below 0.2 and 38 above 0.8, with only 29 in between — so the classification threshold is not delicate.

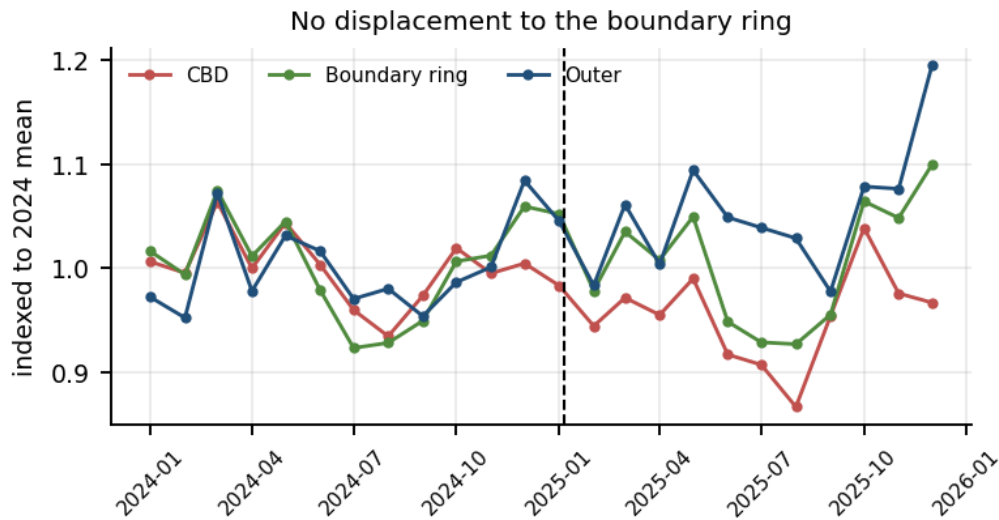


Figure A2. Trip volume by zone segment, indexed to each segment's 2024 mean. The boundary ring grew 0.79% against 5.26% for outer zones, which is inconsistent with riders stopping short of the cordon to avoid the charge.

Physical plan (abridged). Catalyst collapsed six chained projections into two whole-stage code-generated scans, pushed the zone predicate into the Parquet reader as an INSET filter, and placed the exchange as the single shuffle boundary:

```
== Physical Plan ==
AdaptiveSparkPlan isFinalPlan=false
+- HashAggregate(keys=[PULocationID#7, _groupingexpression#3145],
  functions=[count(1), avg(wait_s#343L)])
  +- Exchange hashpartitioning(PULocationID#7, _groupingexpression#3145, 96),
    ENSURE_REQUIREMENTS, [plan_id=1044]
    +- HashAggregate(keys=[...], functions=[partial_count(1), partial_avg(...)])
      +- InMemoryRelation [...], StorageLevel(disk, memory, deserialized, 1 replicas)
        +- *(1) ColumnarToRow
          +- FileScan parquet [...] PushedFilters: [IsNotNull(trip_miles), ...]
```

Full plan: output/p2/physical_plan_groupby.txt in the repository.

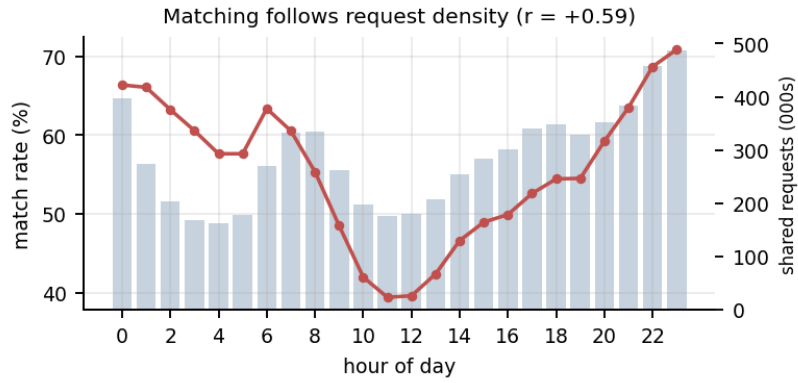


Figure A3. Shared-ride match rate against request volume by hour. Matching succeeds where demand is thick and collapses at midday.

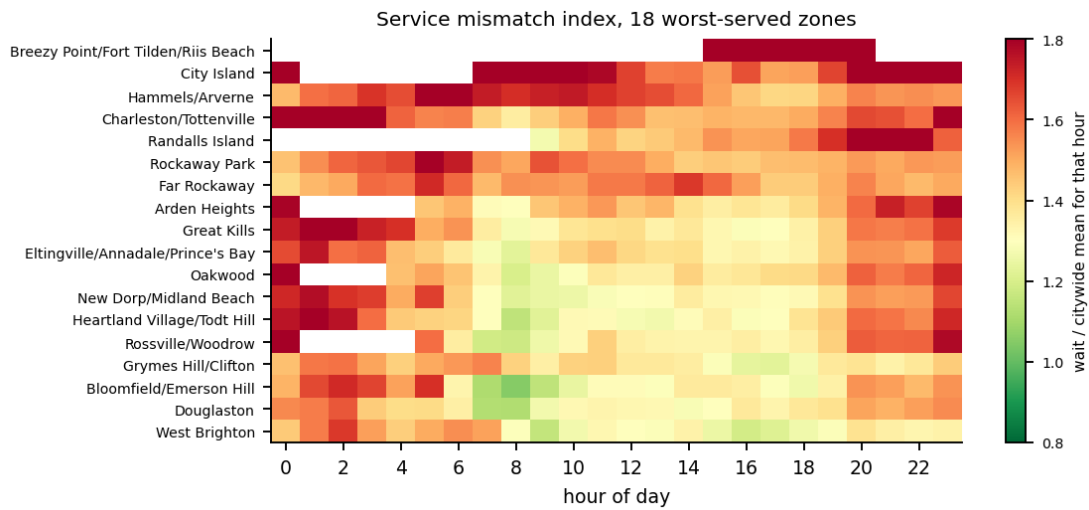


Figure A4. Mismatch index by zone and hour for the 18 worst-served zones. Values above 1.0 indicate waits longer than the citywide mean for that hour.

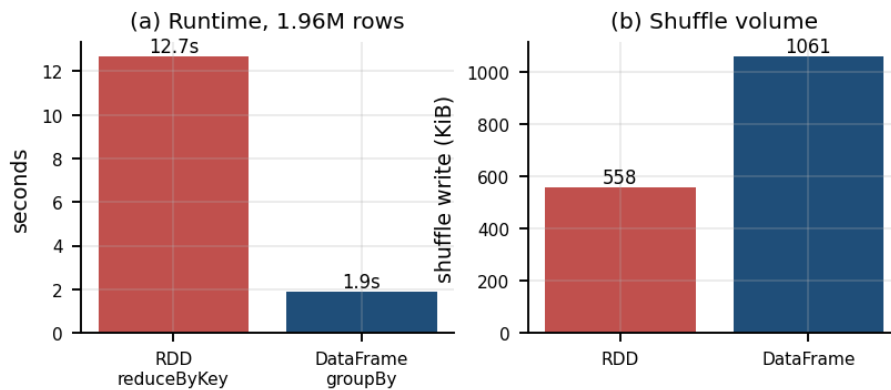


Figure A5. The RDD implementation shuffled less data than the DataFrame implementation yet ran 6.56 $\times$ slower, isolating the JVM-to-Python process boundary as the cost.